

B28: SQL SuperHero Program : Session 20

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
 Bcc: satishhattekar1997@gmail.com

17 March 2025 at 21:30

Decode and case

DECODE and CASE statements in database both provide a conditional construct, of this form:

if A = n1 then A1
 else if A = n2 then A2
 else X

Decode will work like if then else.

**Select Region_Id, decode(region_name, 'Europe','EU','Asia','AS','Americas','AM', 'Middle East and Africa','EMEA')
 Region_Code
 From Regions;**

```

1 select region_id, region_name,
2 decode(REGION_NAME, 'Europe', 'EU', 'Americas', 'AM', 'Asia', 'AS', 'Middle East and Africa', 'EMEA') As Region_Code
3 from hr.regions
4

```

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	EU
2	Americas	AM
3	Asia	AS
4	Middle East and Africa	EMEA

Download CSV

4 rows selected.

Decode function will work as if then else. It works like decode(value,<if this>,<then this>,<if this> <then this>.....);

**Select Region_Id, decode(region_name, 'Europe','EU','Americas','AM', 'Other World') Region_Code
 From Regions;**

**Select Region_Id, region_name, decode(region_name, 'Europe','EU','Americas','AM', 'Other World') Region_Code
 From Regions;**

the table is not altered, just a new column is displayed.

```
select region_id, region_name, decode(REGION_NAME, 'Europe', 'EU', 'Americas', 'AM', 'vishal') Region_Code
from hr.regions
```

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	EU
2	Americas	AM
3	Asia	vishal
4	Middle East and Africa	vishal

4 rows selected.

else

pair pair

if Region name = 'EUROPE'
then EU

```
select Region_id, Region_name, (Case When region_name = 'Europe' Then 'EU'
When region_name = 'Americas' Then 'AM'
Else 'Other World'
End ) Region_Code
From Regions;
```

```
select region_id, region_name,
case when REGION_NAME = 'Europe' then 'EU'
when REGION_NAME = 'Americas' then 'AM'
when REGION_NAME = 'Asia' then 'AS'
when REGION_NAME = 'Middle East and Africa' then 'EMEA'
End As region_code
from hr.regions
```

Case start with
CASE
Ends with
END

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	EU
2	Americas	AM
3	Asia	AS
4	Middle East and Africa	EMEA

4 rows selected.

Difference between Case And decode:

1. Decode will only work with equal to, means compare the exact value but case is more flexible and strong function and it gives us flexibility to compare 2 data
2. Case is same like decode . case is also work as if then else. everything what decode can do, case will do too. there is a lot what case can do and decode can not do.
3. case is a statement and decode is a function.

length --> is a function, which gives you length of the string

```
select Region_id, region_name, length(region_name) from regions;
```

```
select R.*, case when length(region_name) > 6 then 'Big world'
when length(region_name) < 6 then 'small world'
end As region_code
from hr.regions R
```

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	-
2	Americas	Big world
3	Asia	small world
4	Middle East and Africa	Big world

4 rows selected.

If you see the above output, Region_Code is NULL for Europe . Why??? Because when we compare we just say Length >6 and also length < 6. But length('Europe') = 6, so it was just gave null. if we add this as below:

```
select R.*, case when length(region_name) > 6 then 'Big world'
when length(region_name) <= 6 then 'small world'
end As region_code
from hr.regions R
```

→ <= both compared

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	small world
2	Americas	Big world
3	Asia	small world
4	Middle East and Africa	Big world

4 rows selected.

4. IN clause supported by case not by decode. because decode will take only one value to compare, and case is flexible:

supported.

```
select R.*, case when region_name in ('Europe','Americas') then 'EU'
when region_name in ('Asia','Middle East and Africa') then 'AS'
end As region_code
from hr.regions R
```

REGION_ID	REGION_NAME	REGION_CODE
1	Europe	EU
2	Americas	EU
3	Asia	AS
4	Middle East and Africa	AS

4 rows selected.

Case is a statement. we can use it exactly like a sql statement. Multiple check in one statement, not possible in decode.

```
CASE (expression)
✓ WHEN <value1> THEN action_block1;
  WHEN <value2> THEN action_block2;
  WHEN <value3> THEN action_block3;
  ELSE action_block_default;
END CASE;
```

Every when & then is one statement

1. In the above syntax, the expression will return a value that could be of any type (variable, number, etc.).
2. Each 'WHEN' clause is treated as an alternatives which have <value> and <action_block>.
3. The 'WHEN' clause which matches the value as that of the expression will be selected, and the corresponding <action_block> will be executed.
4. 'ELSE' block is optional which hold the <action_block_default> that needs to be executed when none of the alternatives match the expression value.
5. The 'END' marks the end of the CASE statement, and it is a mandatory part of the CASE.

=====

Write a sql to print employee_id, salary and if employee salary is more than 24000 the print EXCELLENT, If salary is between 15000 and 20000 then V GOOD, If between 10000 and 15000 then GOOD and if less than 10000

then Bad.

Understand this sql, read it again. Is it if then else, is the output depends on some condition, is the output depends on some input column as conditional construct? YES, it means CASE is the solution

Condition ←

```
select employee_id, salary, (case when SALARY > 20000 then 'Excellent'
                                when SALARY > 15000 and salary < 20000 then 'v good'
                                when SALARY > 10000 and salary < 15000 then 'good'
                                when SALARY < 10000 then 'bad'
                                end )salary_analysis from hr.employees
```

EMPLOYEE_ID	SALARY	SALARY_ANALYSIS
100	24000	Excellent
101	17000	v good
102	17000	v good
103	9000	bad
104	6000	bad
105	4800	bad

Every When Condition is a statement, once anyone satisfied, it will not check the next one.

```
select employee_id, salary, (case when SALARY > 20000 then 'Excellent'
                                when SALARY between 15000 and 20000 then 'v good'
                                when SALARY between 16000 and 20000 then 'good'
                                else 'bad'
                                end )salary_analysis from hr.employees
```

EMPLOYEE_ID	SALARY	SALARY_ANALYSIS
100	24000	Excellent
101	17000	v good
102	17000	v good
103	9000	bad
104	6000	bad

First satisfy
not
check next

If you see the above example, salary 17000 actually satisfy both conditions, but case will print the first one . As it satisfies one condition, it will not check others. Let em switch:

```

select employee_id, salary, ( case
when salary > 20000 then 'EXCELLENT'
when salary between 16000 and 17500 then 'gOOD'
when salary between 15000 and 18000 then 'V gOOD'
when salary < 10000 then 'BAD'
End ) salary_analysis from hr.employees

```

EMPLOYEE_ID	SALARY	SALARY_ANALYSIS
100	24000	EXCELLENT
101	17000	gOOD
102	17000	gOOD
103	9000	BAD
104	6000	BAD
105	4800	BAD

FLFS

↓

FIRST come

FIRST SERVE

Handling NULL --> Very Important Question

```
select decode(null, null, 'NULL', 'NOT NULL') null_test from dual;
```

--> NULL

```

select (case null
  when null then 'NULL'
  else 'NOT NULL'
end) null_test
from dual;

```

--> NOT NULL

```

select (case
  when null is null then 'NULL'
  else 'NOT NULL'
end) null_test
from dual

```

--> NULL

```
select decode(null, null, 0, 1) null_test from dual;
```

```
1 select decode(null, null, 0, 1) null_test from dual;  
2 |
```

if null is null then 0
else 1.

NULL_TEST

0

[Download CSV](#)

null can be compare by
IS operator

```
1 select (case null  
2         when null then 0  
3         else 1  
4         end) null_test  
5 from dual;
```

null = null then
0

NULL_TEST

1

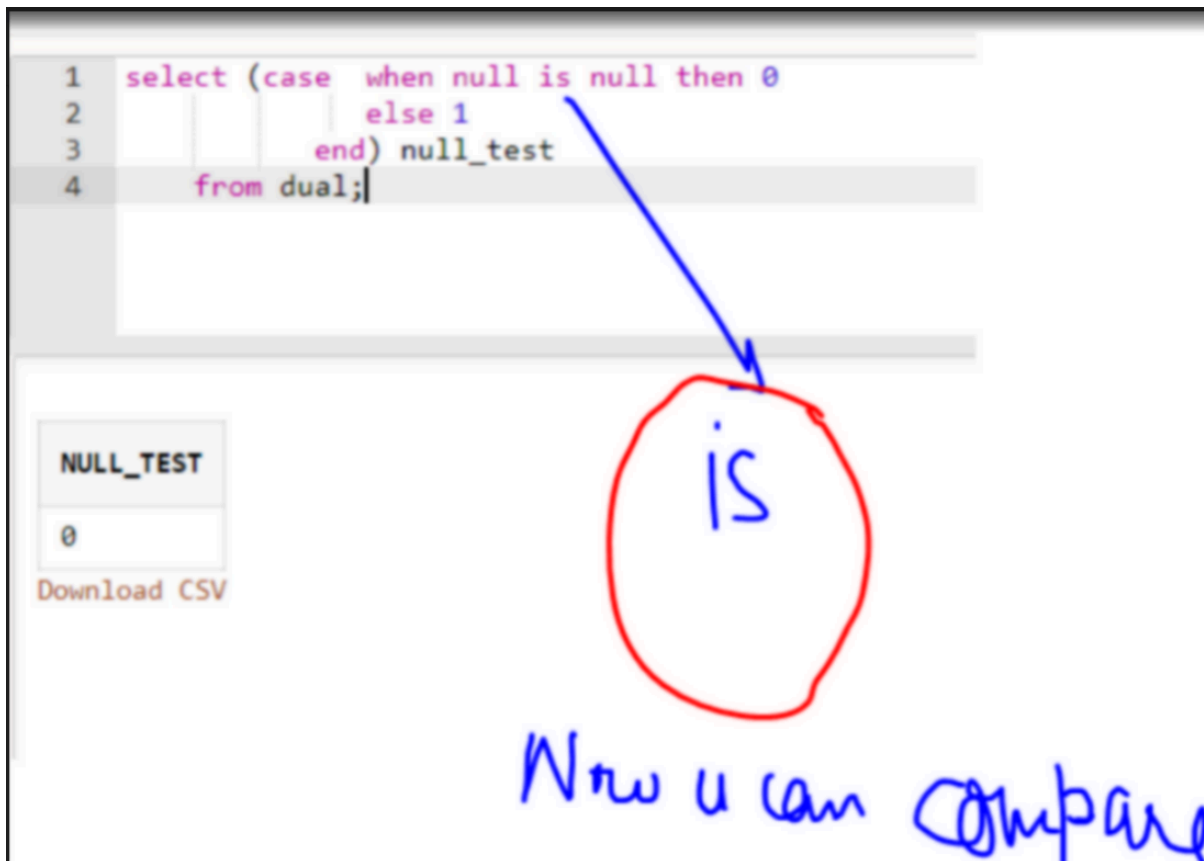
[Download CSV](#)

it fails

```

1 select (case when null is null then 0
2           else 1
3         end) null_test
4 from dual;

```



NULL_TEST
 0
[Download CSV](#)

Now u can compare

```

select decode(2,1,1,
              '2','2',
              '3') t
from dual;

```

T

2

```

select case 2 when 1 then '1'
           when '2' then '2'
           else '3'
        end
from dual;
when '2' then '2'
*

```

ERROR at line 2: number to value. Case is matching number to number and varchar to varchar


```
select decode (2,1,1,'2','2',3) from dual
```

```
DECODE(2,1,1,'2','2',3)
```

```
2
```

it match here

```
select case 2 when 1 then '1'
       when '2' then '2'
else '3'
end from dual
```

→ Here gives error

ORA-00932: inconsistent datatypes: expected NUMBER got CHAR

More Details: <https://docs.oracle.com/error-help/db/ora-00932>

This is also a difference between case and decode. Case wants same datatype to compare.

The diagram illustrates the difference between DECODE and CASE statements. It shows two examples of DECODE and one example of CASE, with handwritten annotations explaining their behavior.

Example 1: DECODE

```
select decode (5,4,5,6,7,8,4,5) from dual
```

Result: 5

Annotations: A handwritten arrow labeled "value" points to the first argument (5). Red arrows labeled "Pair" point to the subsequent pairs of arguments (4,5), (6,7), and (8,4). A red arrow labeled "else" points to the final argument (5).

Handwritten notes to the right:

- if 5=4 then 5 X
- 5=6 then 7 X
- 5=8 then 4 X
- else 5 ✓

Example 2: DECODE

```
select decode (5,4,5,6,7,5,8,5) from dual
```

Result: 8

Annotations: A handwritten arrow labeled "value" points to the first argument (5). Red arrows labeled "Pair" point to the subsequent pairs of arguments (4,5), (6,7), and (8,5). A red arrow labeled "else" points to the final argument (5).

Handwritten notes to the right:

- if 5=4 then 5 X
- 5=6 then 7 X
- 5=5 then 8 ✓
- else 5

Example 3: CASE

```
select case 2 when 1 then '1'
       when '2' then '2'
else '3'
end from dual
```

Question: can we write decode within decode or case within case. Like multiple check ?

Yes.

```
SELECT DECODE (2, 2, DECODE(3, 7, 2, 4, 5, 6)) FROM DUAL;
```

SQL Worksheet

1 `SELECT DECODE (2, 2, DECODE(3, 7, 2, 4, 5, 6)) FROM DUAL`

DECODE(2,2,DECODE(3,7,2,4,5,6))

6

Download CSV

Pair - if + then

Pair

else

if + then

In this example

In the first DECODE 2 is the expression
the next 2 is what the first 2 will be compared to
the next DECODE is the result if 2 = 2

In the second DECODE 3 is the expression

It is compared to the 7 and the 4.

The results could be 2 or 5

Finally if the 3 doesn't match the 7 or the 4 then 6 is the result.

```

-----
IF 2 = 2 THEN
  IF 3 = 7 THEN
    2
  ELSIF 3 = 4 THEN
    5
  ELSE 6
END IF;

```

Pair

↓

select decode(2,decode(4,5,6,4,3,4),4,8) from dual

DECODE(2,DECODE(4,5,6,4,3,4),4,8)
8

if 2 = decode(4,5,6,4,3,4)
 then 4
 else 8

select decode(2,decode(4,5,6,4,2,4),4,8) from dual

Pair

↓

DECODE(2,DECODE(4,5,6,4,2,4),4,8)
4

if 2 = decode(4,5,6,4,2,4)
 then 4
 else 8

if 4 = 5
 then 6
 if 4 = 4
 then 2
 else 4